

중개형 ISA 계좌 서비스

Table of Contents

1

ISA 계좌

(1) ISA 계좌 개념 (2) 중개형 ISA 계좌

2

DB 설계

(1) 비즈니스 요구사항 (2) ERD 설계

3

SQL Query

(1) 기본 쿼리 (2) 심화 쿼리 (3) 인덱스와 뷰

ISA 계좌

-
- 1 ISA 계좌 개념
 - 2 중개형 ISA 계좌

ISA 계좌 개념

Individual Savings Account

ISA 계좌(Individual Savings Account)란?

하나의 계좌로 다양한 금융상품을 관리하면서, 절세혜택도 누릴 수 있는 통합계좌



ISA 계좌 개념

Individual Savings Account

ISA 계좌(Individual Savings Account)란?

유형	일반형	서민형	농어민
가입요건	만 19세 이상 또는 직전연도 근로 소득이 있는 만 15세~19세 미만 대한민국 거주자	직전연도 총급여 5천만원 또는 종합소득 3천8백만원 이하 거주자	직전연도 종합소득 3천8백만원 이하 농어민 거주자
비과세한도	200만원	400만원	400만원
한도초과시	9.9% 저율 분리과세 적용		
의무가입	3년		

[표] ISA계좌란? (국민은행)

ISA 계좌 개념

Individual Savings Account

< 연도별 ISA 총 가입자 수 및 가입금액 현황 >



[표] 연도별 ISA 총 가입자 수 및 가입금액 현황 (금융투자협회)

FT 한국금융신문

'투자 더하기 절세'...증권사 전용 '중개형 ISA' 전성시대

투자과 절세 '두 마리 토끼'를 잡는 투자중개형 ISA(개인종합자산관리계좌)가 가입자 500만명 시대를 예고하고 있다. 중개형 ISA는 증권사에서만 가입...

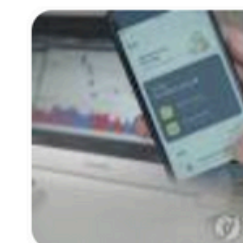
1개월 전

ER 이코노믹리뷰

ISA 가입금액 30조원 돌파 '8년 5개월만'...증권사 통한 가입자 수 비중 84% 육박

2016년 3월에 출시된 개인종합자산관리계좌(ISA)의 전체 가입금액이 도입 8년 5개월 만에 30조원을 돌파했다. ISA는 주식, 펀드, 예금 등 여러 업권의...

2024. 10. 1.



중개형 ISA 계좌

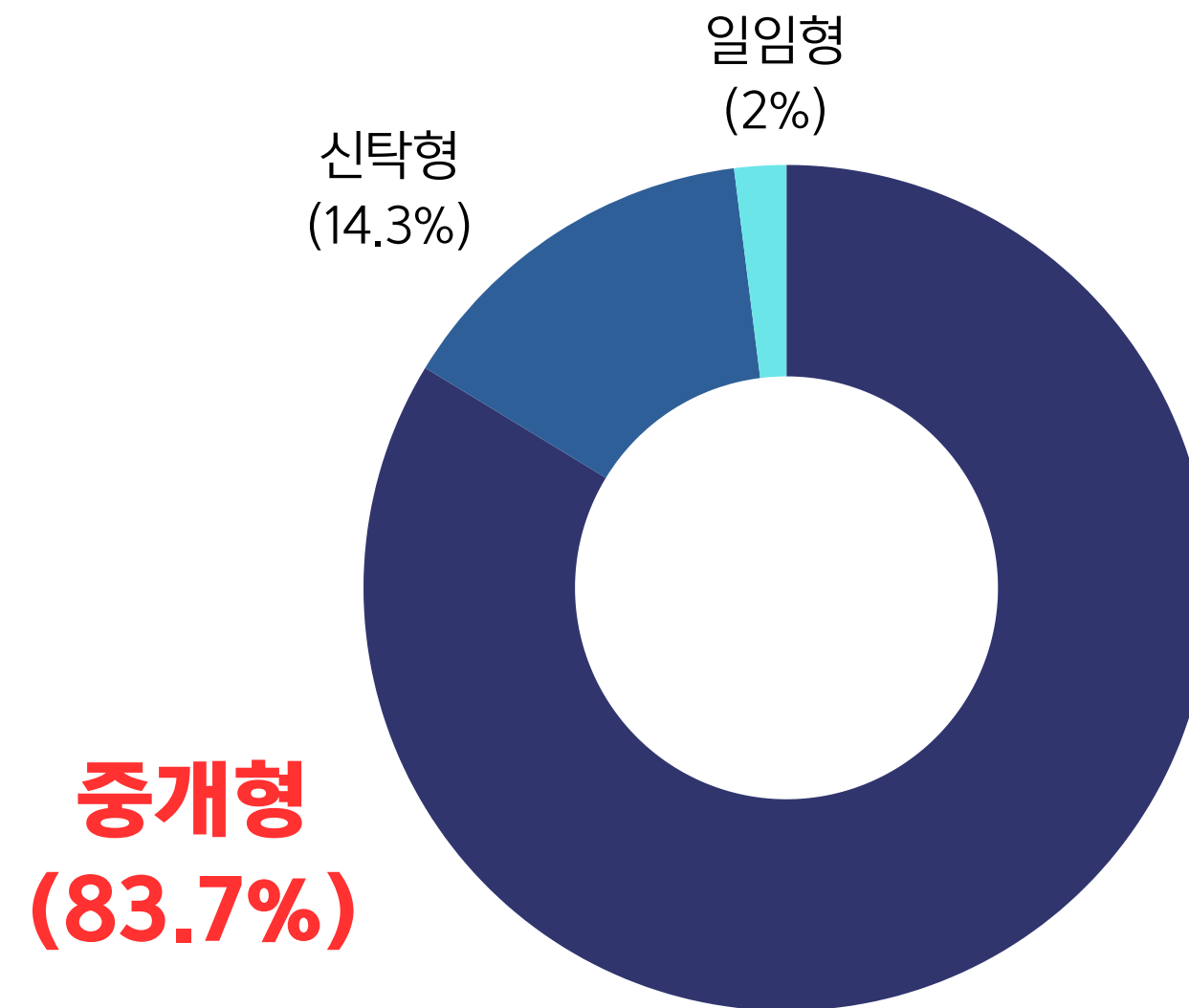
Individual Savings Account

중개형 ISA 계좌?

가입자가 직접 운용 상품을 고르고 관리하는 ISA

중개형 ISA 계좌 특징

- 중개형 ISA는 증권사에서만 계좌 개설 가능함
- 주식 뿐만 아니라 ETF, ELS 등 다양한 상품 거래 가능
- 주식을 직접 담을 수 있는 만큼 **기대수익률** 높고, 그만큼 **절세효과** 기대



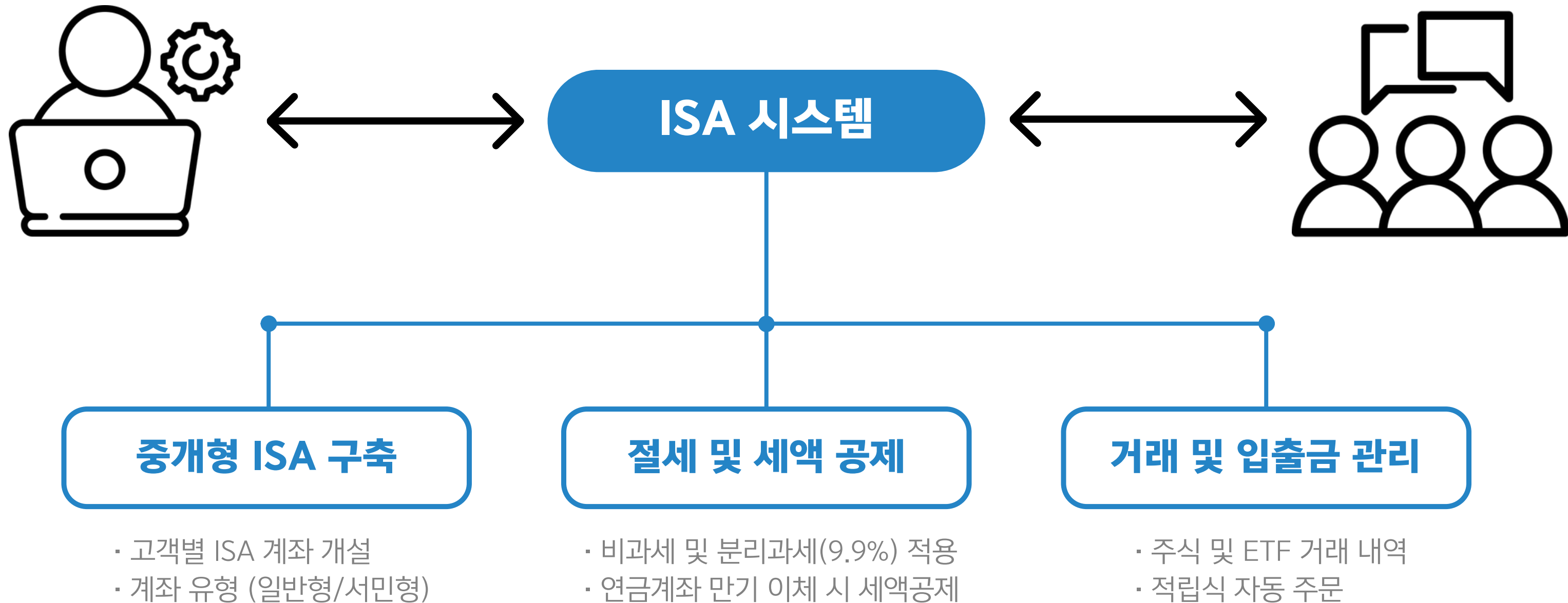
[표] 국내 ISA 계좌 종류별 점유율
(금융투자협회, 2024.12)

DB 설계

-
- 1 비즈니스 요구사항
 - 2 ERD 설계

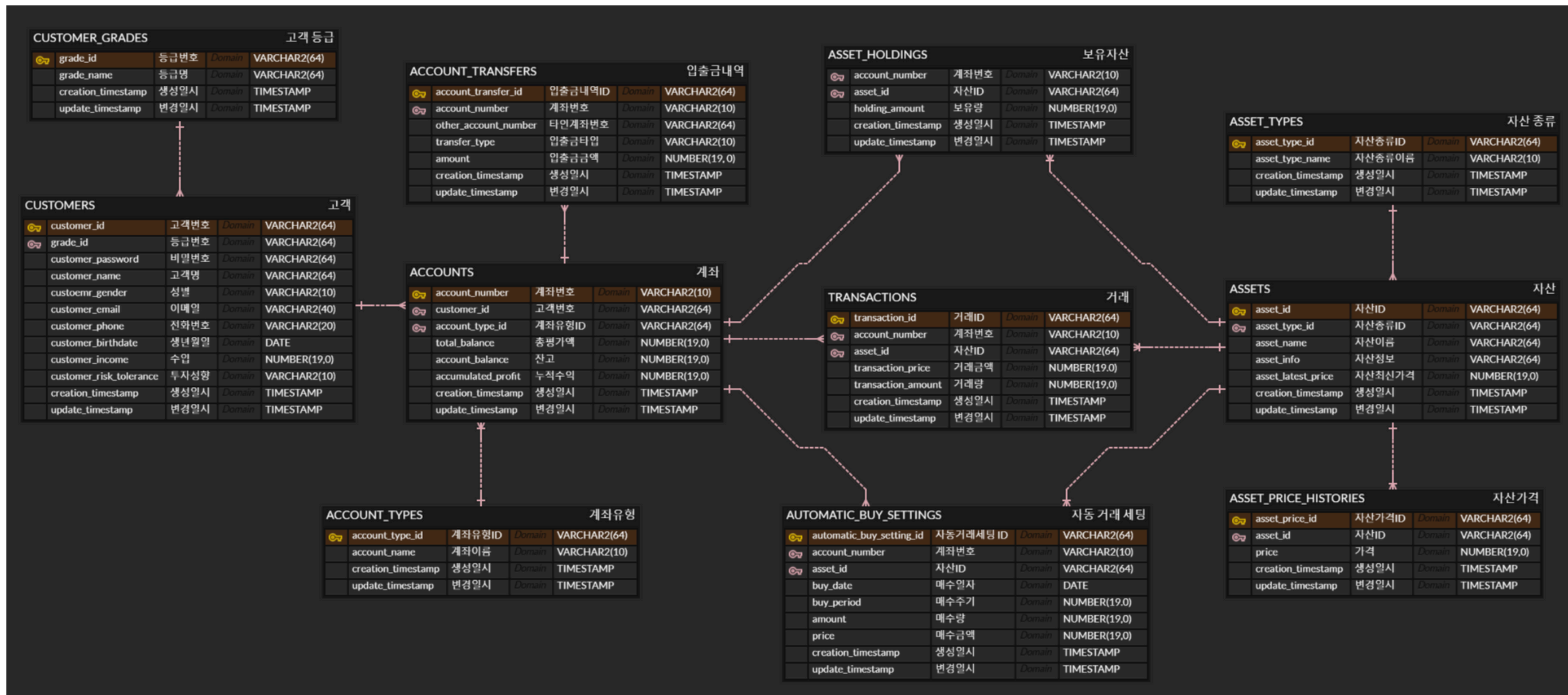
비즈니스 요구사항

Business Requirements



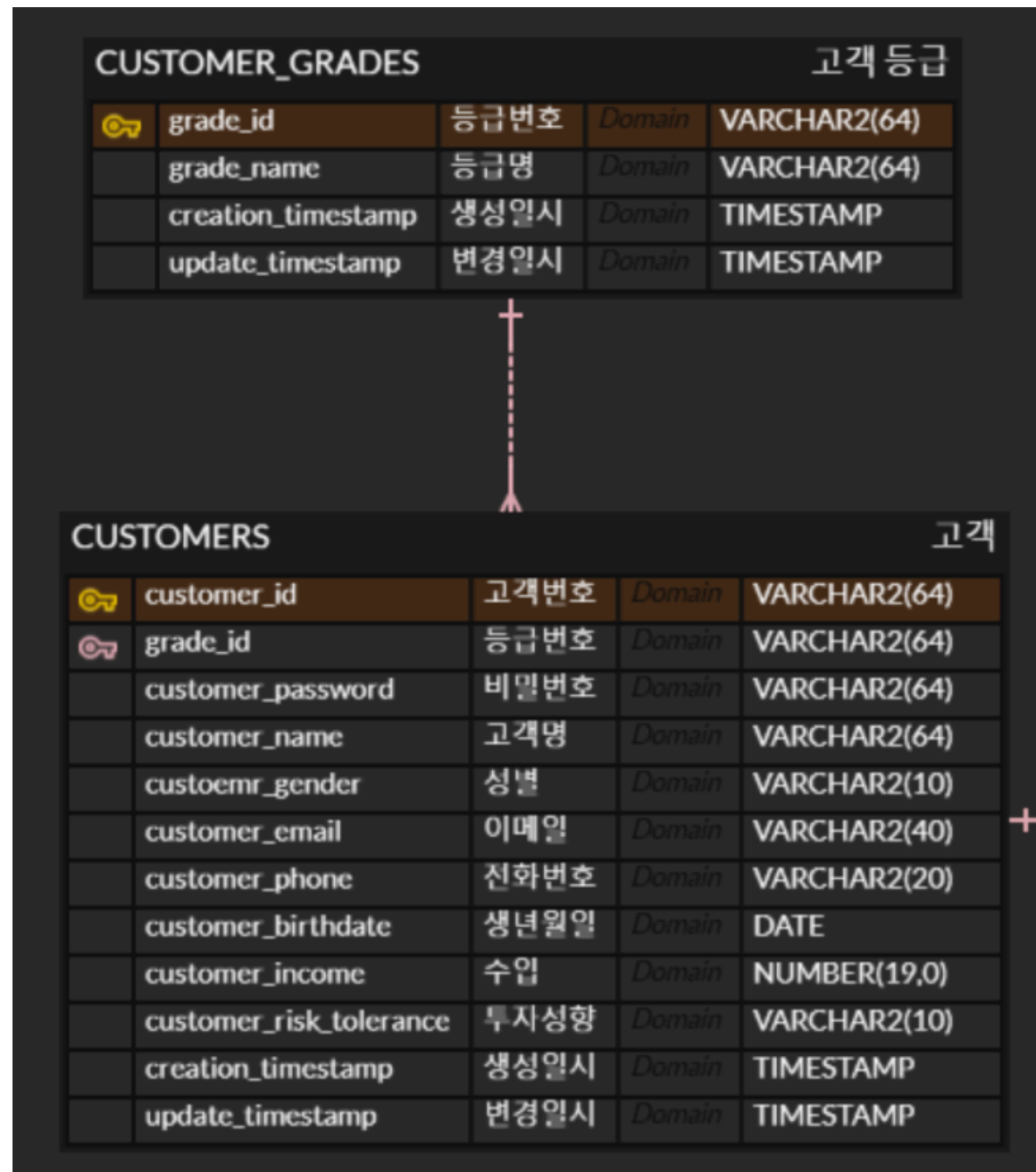
ERD 설계

Entity Relationship Diagram



ERD 설계

Entity Relationship Diagram



CUSTOMERS

증권사 가입 및 계좌 개설 시 등록하는 개인정보

- 수입 : ISA 일반형/ISA 서민형 구분

CUSTOMER_GRADES

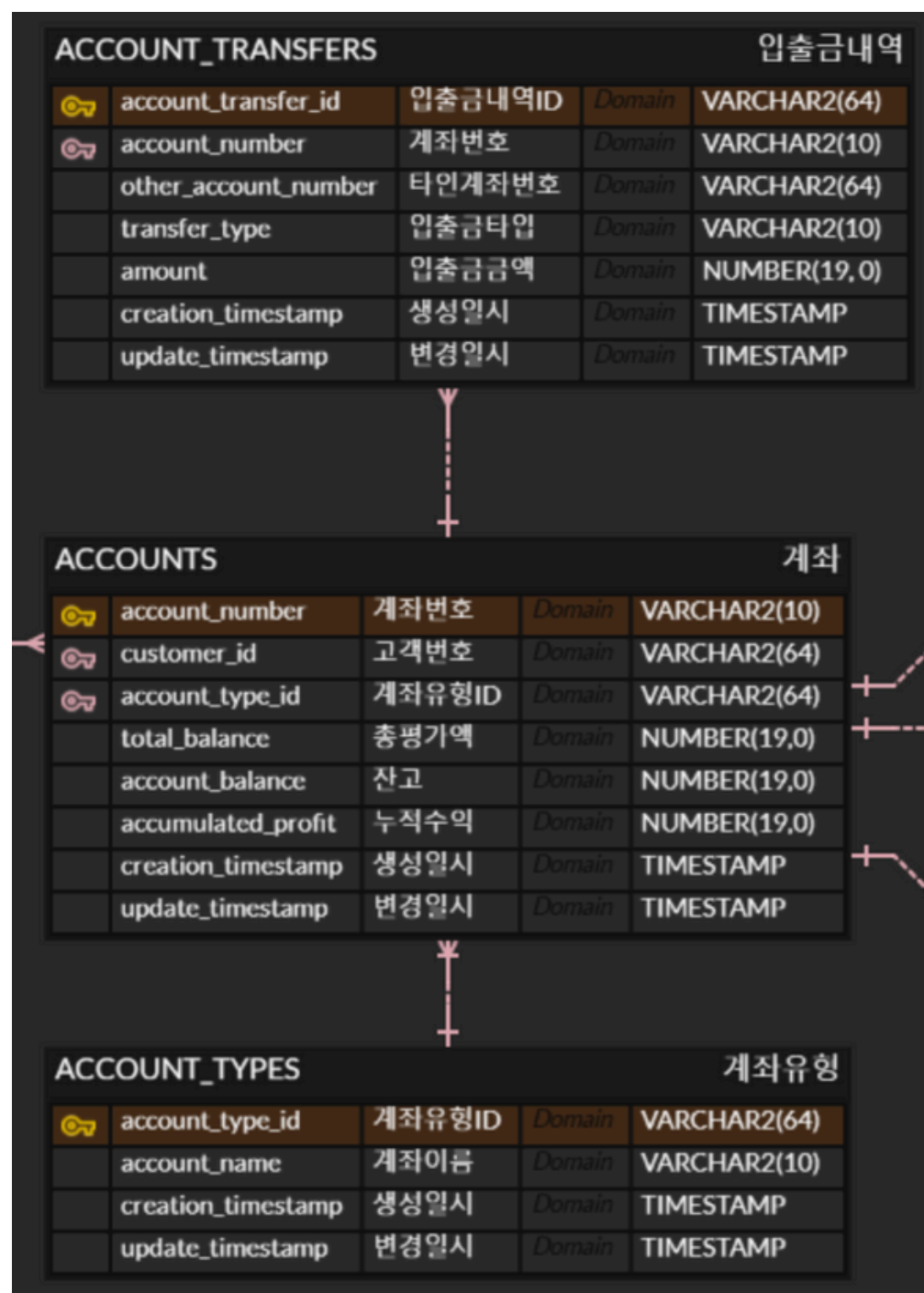
증권거래 시 수수료 우대, 경품 등 각종 혜택 제공

키움증권의 등급에 따라 다음 3단계의 등급으로 구분

- VVIP
- VIP
- General

ERD 설계

Entity Relationship Diagram



ACCOUNTS_TRANSFERS

- 증권거래용 현금 입금 및 수익금 출금을 위한 타 금융기관 입출금 내역

ACCOUNTS

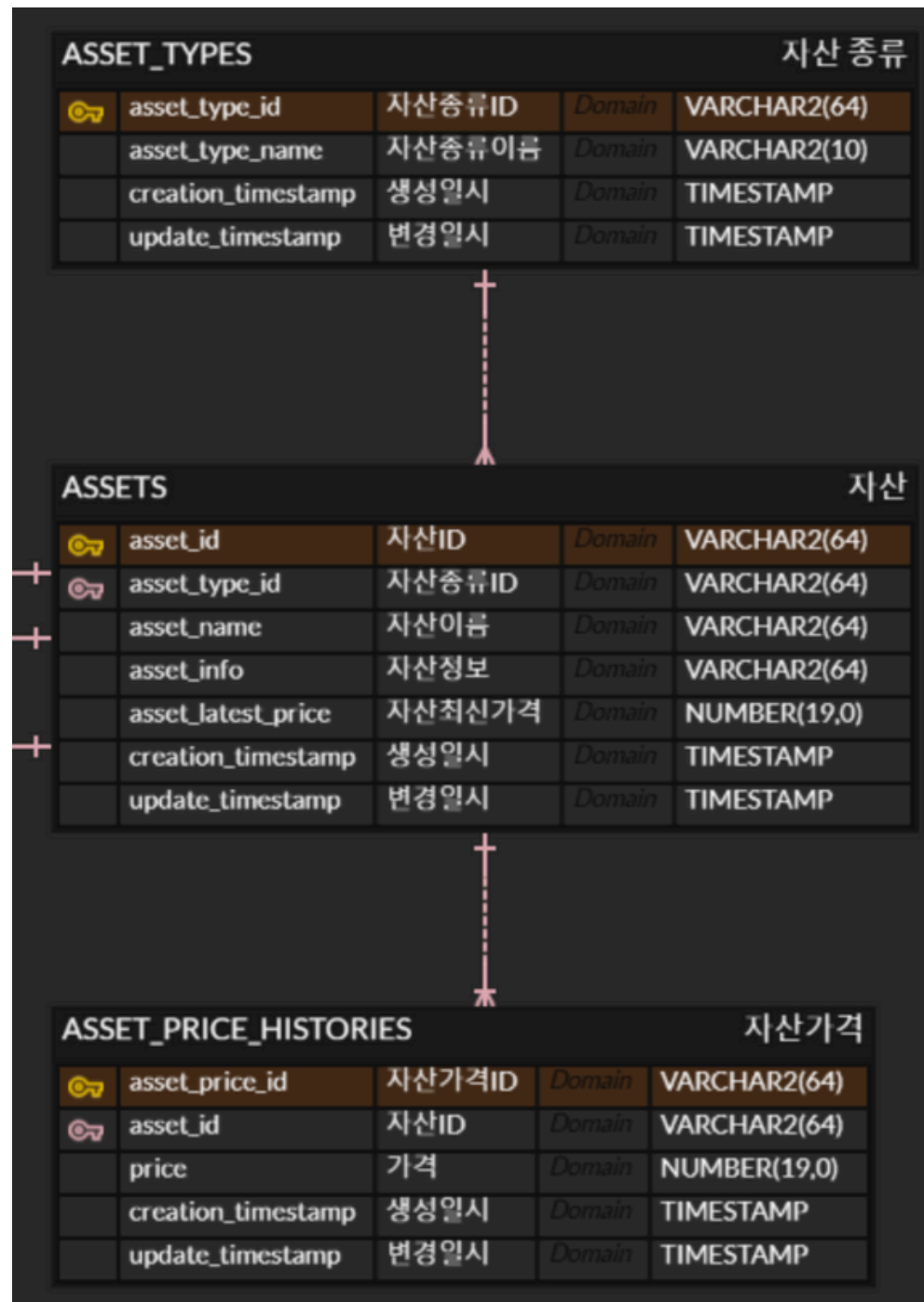
- 고객이 가질 수 있는 계좌 (고객 → 계좌 1:N)
- 총평가액 : 보유 자산 + 예수금
- 잔고 : 예수금
- 누적수익 : 국내 상장 해외 주식 ETF의 거래손익 → 비과세 계산 시 사용

ACCOUNT_TYPES

- 증권사 위탁계좌 형식
- 일반 위탁계좌 / ISA 일반형 / ISA 서민형

ERD 설계

Entity Relationship Diagram



ASSET_TYPES

- 증권거래 및 ISA
- 국내주식 / 해외주식 / 국내ETF / 해외 ETF

ASSETS

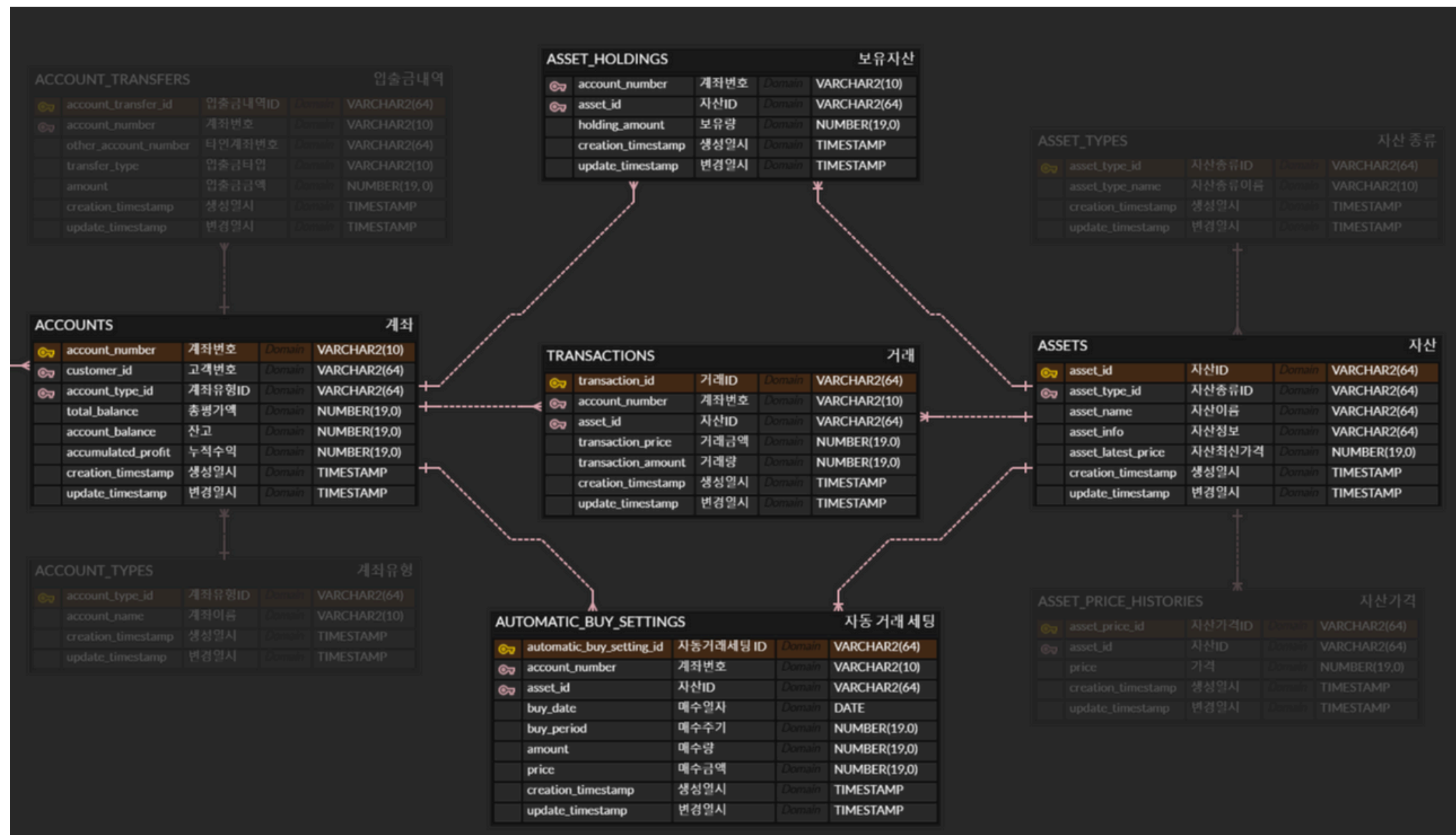
- 증권사에서 거래할 수 있는 자산 목록
- 자산최신가격 : 고객이 가지고 있는 자산의 수익률 및 수익 계산

ASSET_PRICE_HISTORIES

- 자산의 일 단위 종가 계산

ERD 설계

Entity Relationship Diagram



TRANSACTIONS

- 주식 및 ETF 자산의 거래 내역 표시
- ACCOUNTS와 ASSETS 테이블의 중간 테이블

ASSET_HOLDINGS

- 현재 고객이 보유하고 있는 자산 내역을 저장하는 테이블

AUTOMATIC_BUY_SETTINGS

- 자동 적립식 투자 기능
- 일정 매수주기를 설정하고 매수량 및 매수금액에 따른 상품 자동 적립 기능

SQL Query

1 SQL Query

2 인덱스와 뷰

기본 정보 조회

SQL Query

```
-- 고객 정보 조회
SELECT
    customer_id,
    customer_name,
    customer_email,
    customer_phone,
    customer_birthdate,
    customer_income,
    customer_risk_tolerance,
    creation_timestamp,
    update_timestamp
FROM CUSTOMERS
WHERE customer_id = '1';

-- 고객 계좌 정보 조회
SELECT
    a.account_number,
    a.total_balance,
    a.account_balance,
    a.accumulated_profit,
    at.account_name AS account_type_name,
    a.creation_timestamp,
    a.update_timestamp
FROM ACCOUNTS a
JOIN ACCOUNT_TYPES at ON a.account_type_id =
at.account_type_id
WHERE at.account_type_id = '1';

-- 고객 보유 자산 조회
SELECT
    ah.account_number,
    ah.asset_id,
    a.asset_name,
    a.asset_latest_price,
    ah.holding_amount,
    ah.creation_timestamp,
    ah.update_timestamp
FROM ASSET_HOLDINGS ah
JOIN ASSETS a ON ah.asset_id = a.asset_id
WHERE ah.account_number = '1000001';
```

```
-- 특정 주식/상품 정보 조회
SELECT
    asset_id,
    asset_name,
    asset_info,
    asset_latest_price,
    creation_timestamp,
    update_timestamp
FROM ASSETS
WHERE asset_id = '1';

-- 자산 가격 변동 이력 조회
SELECT
    creation_timestamp,
    price
FROM ASSET_PRICE_HISTORIES
WHERE asset_id = '1'
ORDER BY creation_timestamp DESC;

-- 최근 거래 내역 조회
SELECT
    t.transaction_id,
    t.transaction_price,
    t.transaction_amount,
    a.asset_name,
    t.creation_timestamp
FROM TRANSACTIONS t
JOIN ASSETS a ON t.asset_id =
a.asset_id
WHERE t.account_number = '1000001'
ORDER BY t.creation_timestamp DESC;
```

```
-- 자동 매수 설정 조회
SELECT
    abs.automatic_buy_setting_id,
    abs.account_number,
    abs.asset_id,
    a.asset_name,
    abs.buy_date,
    abs.buy_period,
    abs.amount,
    abs.price,
    abs.creation_timestamp
FROM AUTOMATIC_BUY_SETTINGS abs
JOIN ASSETS a ON abs.asset_id =
a.asset_id
WHERE abs.account_number = '1000001';

-- 입출금 내역 조회
SELECT
    at.account_transfer_id,
    at.account_number,
    at.other_account_number,
    at.transfer_type,
    at.amount,
    at.creation_timestamp
FROM ACCOUNT_TRANSFERS at
WHERE at.account_number = '1000001'
ORDER BY at.creation_timestamp DESC;
```


거래 데이터 통계 분석

SQL Query

주식 거래별 평균/총액



```
SELECT
  A.ASSET_NAME,
  ROUND(AVG(T.TRANSACTION_PRICE), 2) AS AVG_PRICE,
  SUM(T.TRANSACTION_PRICE) AS TOTAL_PRICE
FROM TRANSACTIONS T, ASSETS A
WHERE T.ASSET_ID = A.ASSET_ID
GROUP BY A.ASSET_NAME
ORDER BY TOTAL_PRICE DESC;
```

	ASSET_NAME	AVG_PRICE	TOTAL_PRICE
1	기업은행	10000	80000
2	타이거나스닥100	10000	70000
3	현대차	10000	70000
4	KT	10000	70000
5	한미반도체	10000	70000
6	타이거반도체	10000	60000

거래 데이터 통계 분석

SQL Query

국내주식/해외ETF 별 소계



```
SELECT
  NVL(A.ASSET_TYPE_ID, 'Total') ASSET_TYPE_ID,
  ROUND(AVG(T.TRANSACTION_AMOUNT), 2) AVG_AMOUNT,
  AVG(T.TRANSACTION_PRICE) AVG_PRICE
FROM TRANSACTIONS T, ASSETS A
WHERE T.ASSET_ID = A.ASSET_ID
GROUP BY ROLLUP(A.ASSET_TYPE_ID);
```

	ASSET_TYPE_ID	AVG_AMOUNT	AVG_PRICE
1	1	5.29	10000
2	2	5.42	10000
3	Total	5.32	10000

주식 보유 현황

SQL Query

고객별 상품 보유 현황



```
SELECT
  c.customer_id,
  c.customer_name,
  a.account_number,
  ass.asset_name,
  ah.holding_amount,
  ass.asset_latest_price,
  (ah.holding_amount *
  ass.asset_latest_price) AS
  evaluation_amount
FROM CUSTOMERS c
JOIN ACCOUNTS a ON c.customer_id =
a.customer_id
JOIN ASSET_HOLDINGS ah ON
a.account_number = ah.account_number
JOIN ASSETS ass ON ah.asset_id =
ass.asset_id
ORDER BY c.customer_id, ass.asset_name;
```

	CUSTOMER_ID	CUSTOMER_NAME	ACCOUNT_NUMBER	ASSET_NAME	HOLDING_AMOUNT	ASSET_LATEST_PRICE	EVALUATION_AMOUNT
1	1	이종우	1000001	다우기술	22	10000	220000
2	1	이종우	1000001	타이거다우존스	92	12990	1195080
3	1	이종우	1000001	현대차	70	207500	14525000
4	10	이재진	1000010	삼성전자	92	52400	4820800
5	10	이재진	1000010	키움증권	37	124900	4621300
6	10	이재진	1000010	포스코홀딩스	34	10000	340000

주식 보유 현황

SQL Query

고객별 순위 계산



```
SELECT
    RANK() OVER (ORDER BY
        SUM(t.transaction_price *
            t.transaction_amount) DESC) AS
        customer_rank,
        c.customer_id,
        c.customer_name,
        COUNT(t.transaction_id) AS
        total_transactions,
        SUM(t.transaction_price *
            t.transaction_amount) AS
        total_transaction_amount
    FROM CUSTOMERS c
    JOIN ACCOUNTS a ON c.customer_id =
        a.customer_id
    JOIN TRANSACTIONS t ON a.account_number
        = t.account_number
    GROUP BY c.customer_id, c.customer_name
    ORDER BY customer_rank;
```

	CUSTOMER_RANK	CUSTOMER_ID	CUSTOMER_NAME	TOTAL_TRANSACTIONS	TOTAL_TRANSACTION_AMOUNT
1	1	12	임광영	6	470000
2	2	9	신철환	6	440000
3	3	7	반주현	6	400000
4	4	6	김태준	6	380000
5	5	15	김강온	6	360000
6	6	10	이재진	6	340000

주가 변동

SQL Query

특정 종목의 전일 대비 가격변동

```
SELECT
  asset_id,
  creation_timestamp,
  PRICE,
  LAG(PRICE) OVER (
    PARTITION BY asset_id
    ORDER BY creation_timestamp
  ) AS prev_price,
  PRICE - LAG(PRICE) OVER (
    PARTITION BY asset_id
    ORDER BY creation_timestamp
  ) AS price_change
FROM ASSET_PRICE_HISTORIES
WHERE ASSET_ID = '1';
```

	ASSET_ID	CREATION_TIMESTAMP	PRICE	PREV_PRICE	PRICE_CHANGE
1	1	22/01/03 00:00:00.000000000	107000	(null)	(null)
2	1	22/01/04 00:00:00.000000000	107000	107000	0
3	1	22/01/05 00:00:00.000000000	106000	107000	-1000
4	1	22/01/06 00:00:00.000000000	102500	106000	-3500
5	1	22/01/07 00:00:00.000000000	102000	102500	-500
6	1	22/01/10 00:00:00.000000000	102000	102000	0

주가 변동

SQL Query

종목별 52주 최고/최저가 (25년 1월 31일 기준)

```
WITH price_history AS (  
  SELECT  
    asset_id,  
    creation_timestamp,  
    price AS current_price,  
    MAX(price) OVER (  
      PARTITION BY asset_id  
      ORDER BY creation_timestamp  
      RANGE BETWEEN NUMTODSINTERVAL(364, 'DAY') PRECEDING AND CURRENT ROW  
    ) AS high_52_weeks,  
    MIN(price) OVER (  
      PARTITION BY asset_id  
      ORDER BY creation_timestamp  
      RANGE BETWEEN NUMTODSINTERVAL(364, 'DAY') PRECEDING AND CURRENT ROW  
    ) AS low_52_weeks  
  FROM ASSET_PRICE_HISTORIES  
)  
SELECT *  
FROM price_history  
WHERE TRUNC(creation_timestamp) = DATE '2025-01-31';
```

	ASSET_ID	CREATION_TIMESTAMP	CURRENT_PRICE	HIGH_52_WEEKS	LOW_52_WEEKS
1	1	25/01/31 00:00:00.000000000	124900	144600	107700
2	10	25/01/31 00:00:00.000000000	52400	87800	49900
3	11	25/01/31 00:00:00.000000000	17930	24300	17080
4	12	25/01/31 00:00:00.000000000	113300	189000	58800
5	13	25/01/31 00:00:00.000000000	101800	132300	90800
6	14	25/01/31 00:00:00.000000000	128400	194200	99500

주가 변동

SQL Query

특정 종목의 5일 이동평균

```
SELECT
  asset_id,
  creation_timestamp,
  PRICE,
  AVG(PRICE) OVER (
    PARTITION BY asset_id
    ORDER BY creation_timestamp
    ROWS BETWEEN 4 PRECEDING AND CURRENT ROW
  ) AS moving_avg_5d
FROM ASSET_PRICE_HISTORIES
WHERE ASSET_ID = '1';
```

	ASSET_ID	CREATION_TIMESTAMP	PRICE	MOVING_AVG_5D
1	1	22/01/03 00:00:00.000000000	107000	107000
2	1	22/01/04 00:00:00.000000000	107000	107000
3	1	22/01/05 00:00:00.000000000	106000	106666.67
4	1	22/01/06 00:00:00.000000000	102500	105625
5	1	22/01/07 00:00:00.000000000	102000	104900
6	1	22/01/10 00:00:00.000000000	102000	103900

절세 및 세액 공제

SQL Query

ISA 계좌 세액공제 효과 비교

```
SELECT
  CUSTOMER_ID AS "고객ID",
  ACCUMULATED_PROFIT AS "실현이익",
  TAXES AS "ISA 적용 세금",
  NO_ISA AS "일반계좌 세금",
  NO_ISA - TAXES AS "중도해지 시 추가 세금"

FROM (
  SELECT
    C.CUSTOMER_ID,
    A.ACCUMULATED_PROFIT,
    CASE
      WHEN A.ACCOUNT_TYPE_ID = 1 THEN GREATEST(0, FLOOR((A.ACCUMULATED_PROFIT - 2000000) * 0.099)) -- ISA 일반형
      WHEN A.ACCOUNT_TYPE_ID = 2 THEN GREATEST(0, FLOOR((A.ACCUMULATED_PROFIT - 4000000) * 0.099)) -- ISA 서민형
      ELSE GREATEST(0, FLOOR(A.ACCUMULATED_PROFIT * 0.154)) -- 일반계좌
    END AS TAXES,
    GREATEST(0, FLOOR(A.ACCUMULATED_PROFIT * 0.154)) AS NO_ISA
  FROM CUSTOMERS C, ACCOUNTS A
  WHERE C.CUSTOMER_ID = A.CUSTOMER_ID
) S;
```

ISA 계좌를 국내상장 해외시장 ETF를 구매하는 데 자주 사용하는 트렌드를 반영하여 해당 종목만 구매하였다는 가정에 기반하여 계산

절세 및 세액 공제

SQL Query

ISA 계좌 세액공제 효과 비교

	고객ID	실현이익	ISA 적용 세금	일반계좌 세금	중도해지 시 추가 세금
1	1	-2308929	0	0	0
2	2	3306737	129366	509237	379871
3	3	-9054150	0	0	0
4	4	6876618	482785	1058999	576214
5	5	-1669304	0	0	0
6	6	3068030	105734	472476	366742
7	7	-13569304	0	0	0
8	8	1669297	0	257071	257071
9	9	-6120744	0	0	0
10	10	3380559	136675	520606	383931
11	11	-10030797	0	0	0
12	12	2308929	30583	355575	324992
13	13	-2680629	0	0	0
14	14	3379152	136536	520389	383853
15	15	-12791548	0	0	0
16	16	1669311	0	257073	257073

ISA 적용 세금

- ISA 일반형 : 실현이익의 200만원까지 비과세, **차액 9.9% 과세**
- ISA 서민형 : 실현이익의 400만원까지 비과세, **차액 9.9% 과세**

일반계좌 세금

- 국내상장 해외 ETF 종목은 매매차익 또한 **배당소득세 15.4% 과세**

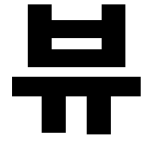
중도해지 시 추가 세금

- 의무가입기간(3년) 내 운용수익금 출금시 **15.4% 일반과세**

인덱스

Index

```
-- 자산 가격 변동 이력 조회 시 자산별로 select 하는 쿼리가 많음.  
asset_price_histories table_index (asset_id)  
  
-- 자동 매수 설정 조회 시 계좌별로 설정된 자동 매수를 select 하는 쿼리가 많음.  
automatic_buy_setting table_index (account_number)  
  
-- 입출금 내역 조회 시 계좌별 입출금 내역을 보여주기 위한 쿼리가 많음.  
account_transfers table_index (account_number)  
  
-- 거래 내역 조회시 계좌 별로 거래 내역을 불러오는 쿼리가 많음.  
transactions table_index (account_number)  
  
-- 자산 가격 변동 이력의 경우 대용량 데이터를 다루는 테이블  
-- order by를 시간 순으로 하는 경우가 많기 때문에  
-- creation_timestamp에 index 설정.  
asset_price_histories table_index (creation_timestamp)
```



View



인덱스와 뷰

```
-- 고객별 거래 요약 뷰
CREATE OR REPLACE VIEW CUSTOMER_TRANSACTION_SUMMARY AS
SELECT
    c.customer_id,
    c.customer_name,
    COUNT(t.transaction_id) AS total_transactions,
    SUM(t.transaction_price * t.transaction_amount) AS
total_transaction_amount,
    MAX(t.creation_timestamp) AS last_transaction_date
FROM CUSTOMERS c
JOIN ACCOUNTS a ON c.customer_id = a.customer_id
JOIN TRANSACTIONS t ON a.account_number = t.account_number
GROUP BY c.customer_id, c.customer_name;
```

```
-- ISA계좌 세액공제 효과 비교 뷰
CREATE OR REPLACE VIEW ISA_TAX_CREDIT_EFFECT AS
SELECT
    CUSTOMER_ID AS "고객ID",
    ACCUMULATED_PROFIT AS "실현이익",
    TAXES AS "ISA 적용 세금",
    NO_ISA AS "일반계좌 세금",
    NO_ISA - TAXES AS "중도해지 시 추가 세금"
FROM
    (SELECT C.CUSTOMER_ID, A.ACCUMULATED_PROFIT,
CASE
    WHEN A.ACCOUNT_TYPE_ID = 1 THEN GREATEST(0, FLOOR((A.ACCUMULATED_PROFIT
- 2000000) * 0.099)) -- ISA 일반형
    WHEN A.ACCOUNT_TYPE_ID = 2 THEN GREATEST(0, FLOOR((A.ACCUMULATED_PROFIT
- 4000000) * 0.099)) -- ISA 서민형
    ELSE GREATEST(0, FLOOR(A.ACCUMULATED_PROFIT * 0.154)) -- 일반계좌
END AS TAXES,
    GREATEST(0, FLOOR(A.ACCUMULATED_PROFIT * 0.154)) AS NO_ISA
FROM CUSTOMERS C, ACCOUNTS A
WHERE C.CUSTOMER_ID = A.CUSTOMER_ID) S;
```

감사합니다